Apartment Rental Management System

Proposal / Part 1 / 26 October 2026

Team: The Design Architects
Members: Ar Bilalli, Dren Hoda, Jon Ymeri and Gent Shala

Purpose and users

We propose a small browser application for reserving entire apartments for short stays. Guests find an apartment for their dates and guest count, make a reservation, view their reservations, and cancel an eligible reservation. Managers maintain their own listings and view reservations for those listings.

In scope	Out of scope
Search by city, dates and capacity. Display a fixed nightly price. Reserve available apartments and retain a price total. Cancel before check-in.	Online payments, refunds, reviews, messaging, maps, dynamic pricing and Airbnb integration.
Managers create, edit and deactivate their own listings. Seeded demo users select guest or manager roles. Save browser data locally.	Real account registration/login, public production hosting, a shared server, simultaneous bookings across devices or tabs.

Proposed environment: HTML/CSS and JavaScript classes, following the Todo labs. One active browser tab. Local storage for persistence and an in-memory mode for repeatable checks. This scope is proposed for instructor feedback; Part 1 contains design rather than implementation.

Public repository: https://github.com/drenhoda1/apartment-rental-management

Rules and responsibility assignments

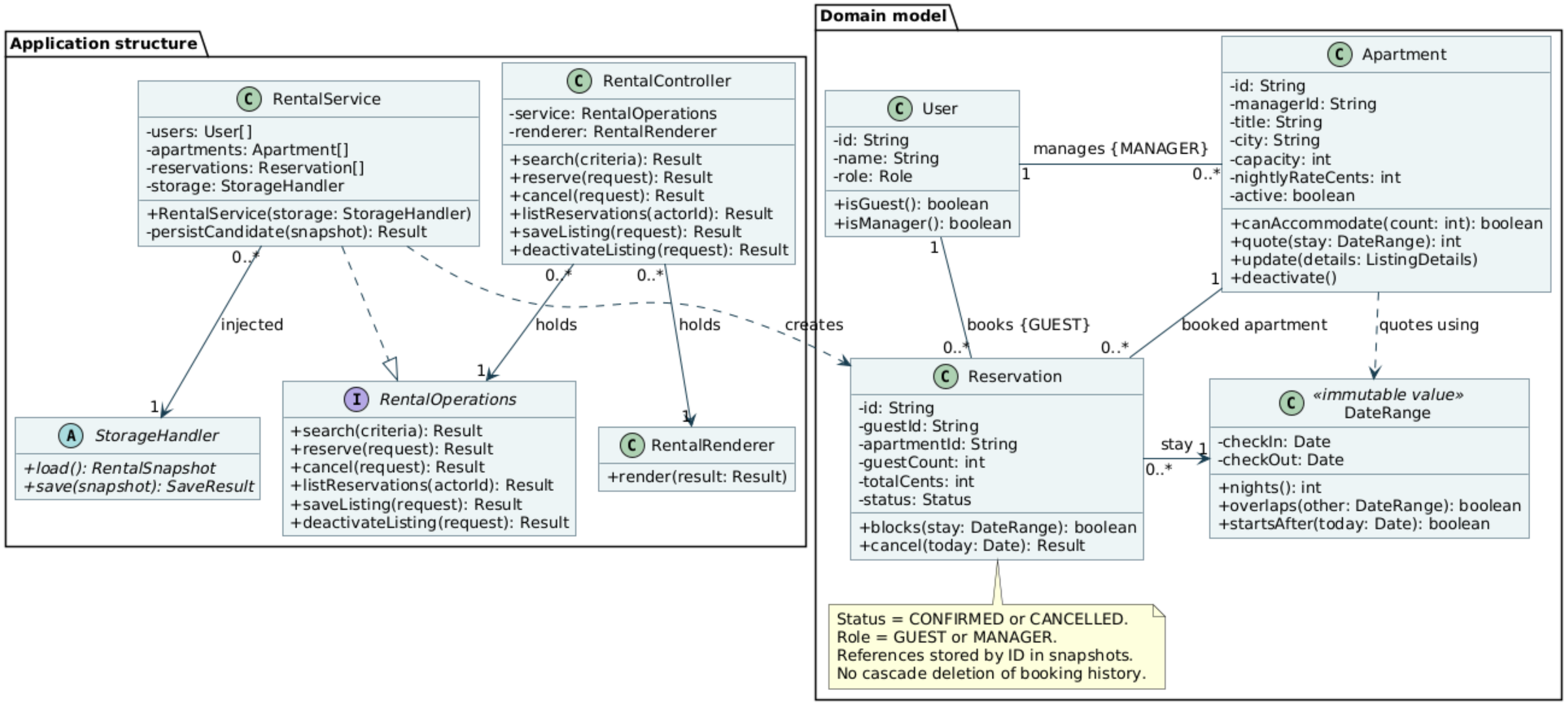
Rule	Design decision
Dates and availability	Check-in is included; checkout is excluded. Checkout must follow check-in, and check-in cannot be in the past. Count calendar nights, not elapsed hours. Only CONFIRMED reservations block dates.
Price and capacity	Guest count must be positive and within apartment capacity. Store EUR amounts as integer cents. Total = nightly rate x nights, captured at booking.
Cancellation and history	Only the guest who owns the reservation can cancel it, while CONFIRMED and before check-in. Set CANCELLED and retain the record. Repeated cancellation is rejected.
Manager actions	Only a manager can create a listing, and only its manager can edit/deactivate it. Deactivation blocks new reservations without erasing history. Existing bookings retain their dates, guest counts and agreed prices.
Save behavior	Prepare an independent candidate state. Save it, then accept it. If storage fails, keep the previous state and display an error.

Example: 10-12 November and 12-14 November do not overlap. A reservation for 11-13 November conflicts with a confirmed 10-12 November stay.

Assigned review/presentation responsibilities: Dren - class diagram and architecture; Ar - scope and search sequence; Jon - reservation sequence and storage risk; Gent - cancellation sequence and logging risk. GitHub commits and reviews will record actual contributions.

Class diagram

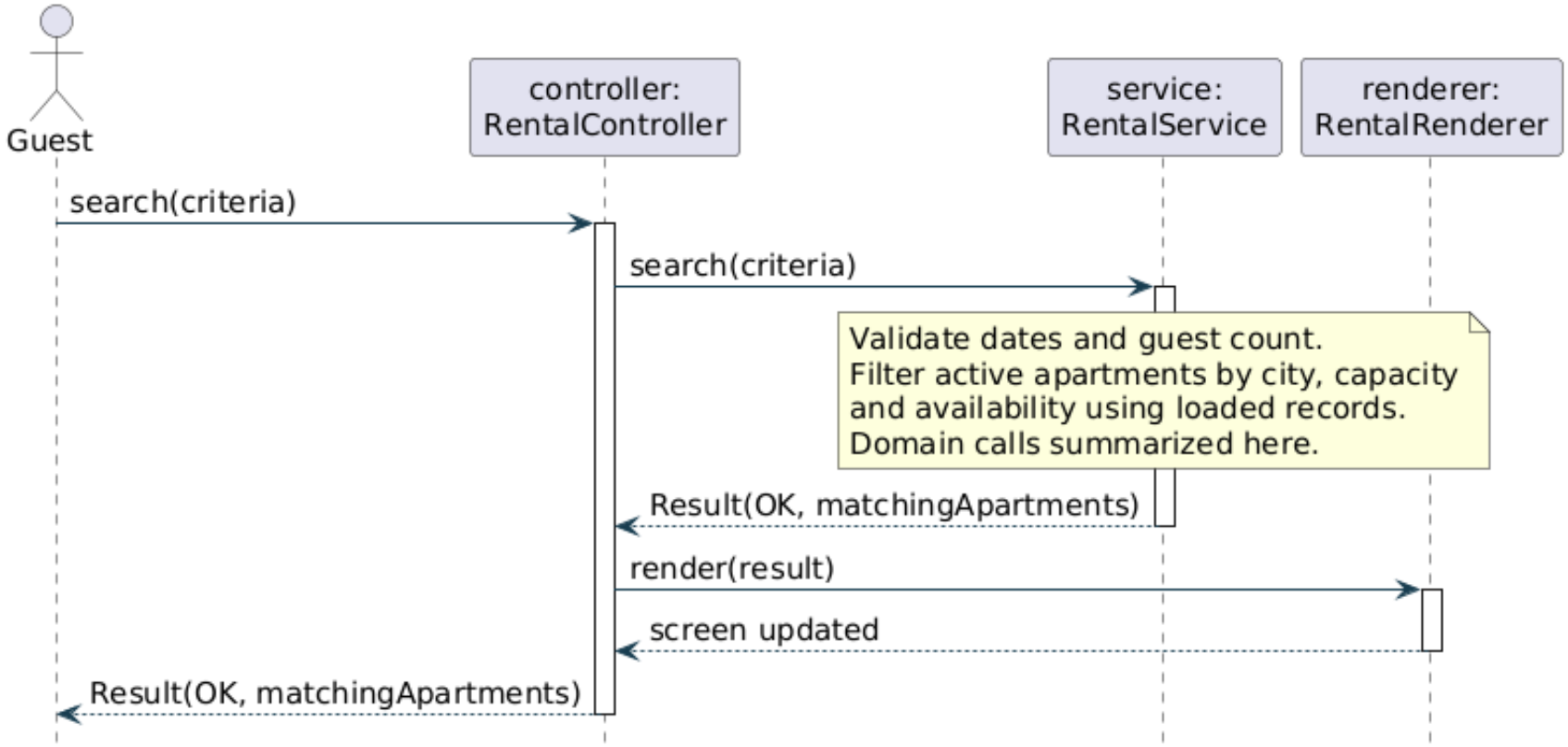
One combined core class diagram. Startup factories and the optional decorator are planned extensions described in the risk section.



RentalService implements the six RentalOperations methods. Its arrays hold the domain objects shown below. Solid links show references/associations. Dashed hollow triangles mean interface implementation; other dashed arrows show dependencies. + public, - private; 0..* means zero or more. There is no cascade deletion of reservation history.

Search apartments

Sequence responsibility: Ar Bilalli / Main success scenario



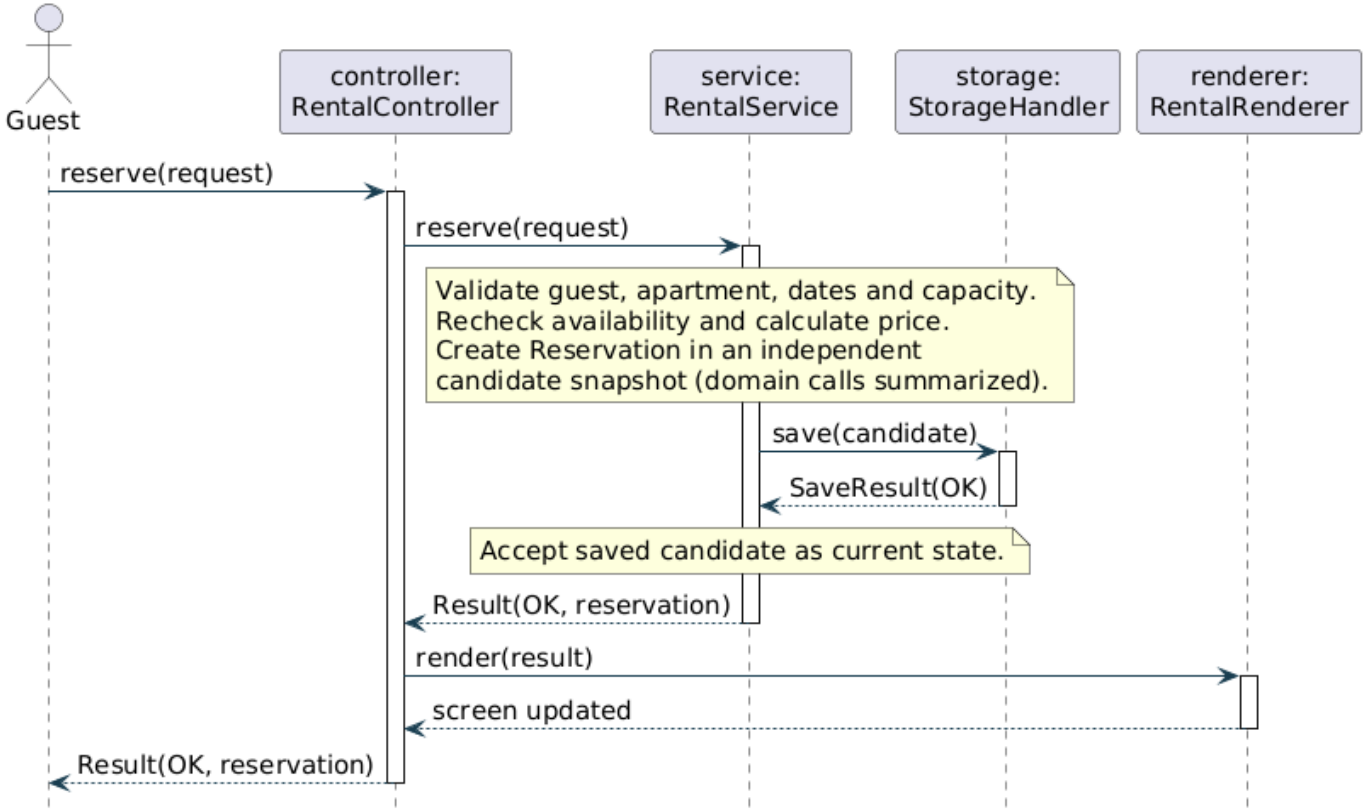
Preconditions: Guest selects city, check-in, checkout and guest count. Valid data has been loaded at startup.

Success guarantee: Matching apartments appear on screen. No stored state changes. **Extensions:** A valid search with no matches returns an empty list. Invalid dates/count return INVALID_INPUT. Search does not save data.

Solid arrows are synchronous calls, dashed arrows are returns, and thin rectangles are activations. Domain calls inside notes are summarized to keep the sequence readable. The renderer updates the screen; the final Result represents the response to the initiating action.

Make a reservation

Sequence responsibility: Jon Ymeri / Main success scenario



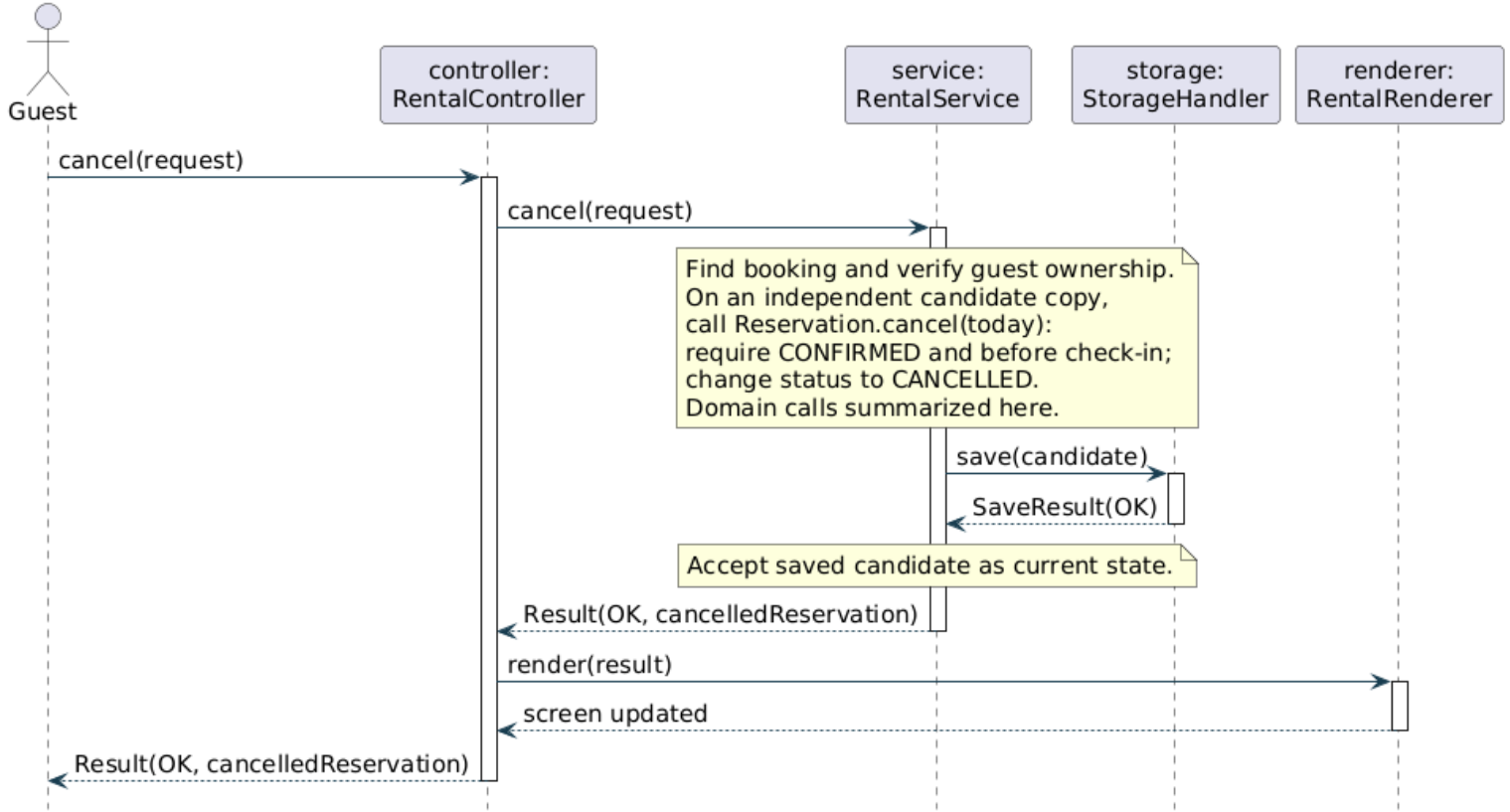
Preconditions: A demo guest selects an apartment and supplies valid dates and guest count. The apartment is available and the save succeeds.

Success guarantee: One confirmed reservation is saved and displayed. Local-storage mode survives page reload. **Extensions:** An overlap returns UNAVAILABLE. Invalid data returns INVALID_INPUT; wrong role returns FORBIDDEN; unknown apartment returns NOT_FOUND. A save error returns STORAGE_ERROR without accepting the candidate.

Solid arrows are synchronous calls, dashed arrows are returns, and thin rectangles are activations. Domain calls inside notes are summarized to keep the sequence readable. The renderer updates the screen; the final Result represents the response to the initiating action.

Cancel a reservation

Sequence responsibility: Gent Shala / Main success scenario



Preconditions: A demo guest selects their own CONFIRMED reservation before check-in. Data is loaded and saving succeeds.

Success guarantee: The saved booking is CANCELLED, its history remains, and its dates no longer block availability. **Extensions:** Wrong role/owner returns FORBIDDEN. Unknown reservation returns NOT_FOUND. A cancelled reservation or check-in reached returns INVALID_STATE. A save failure preserves the original state.

Solid arrows are synchronous calls, dashed arrows are returns, and thin rectangles are activations. Domain calls inside notes are summarized to keep the sequence readable. The renderer updates the screen; the final Result represents the response to the initiating action.

Design risks and planned patterns

Anticipated risk	Planned pattern and application	Trade-off
Storage choice spreads into booking rules. Persistent demo data and temporary checks need different storage behavior.	Factory Method (Weeks 5-6). RentalFactory.createService() calls the overridable createStorage() and injects its StorageHandler into RentalService. LocalRentalFactory returns LocalStorageHandler; MemoryRentalFactory returns InMemoryStorageHandler. Booking rules use the abstraction.	Extra factory and handler classes. Two actual storage modes justify the proposal. Startup still selects the desired factory.
Diagnostic logging clutters service methods and becomes difficult to enable or remove.	Decorator (Week 7). LoggingRentalServiceDecorator implements RentalOperations, holds a RentalOperations reference and forwards all six methods exactly once. Log mutation action, outcome and record ID; preserve results and errors. RentalController can use the plain service or the wrapper.	Forwarding methods must stay aligned with the interface. Logging failures must not change the reported outcome of a saved booking.
Overlapping bookings or false success after a failed save.	GRASP Information Expert puts date overlap in DateRange and booking status in Reservation. RentalService rechecks availability, saves an independent candidate, then accepts it.	These rules protect the single-tab demo. Factory Method and Decorator do not solve cross-device concurrency; a future shared server needs atomic check-and-save.

These patterns are design proposals for instructor feedback. The core UML omits startup construction and the optional logging wrapper to focus on business interactions. We defer Builder, Prototype, Singleton and Adapter because the current scope does not require their specific problems to be solved.

Course alignment and planned checks

Course material	Application in this proposal
Week 2 UML lecture and lab	Classes with attributes/operations, associations and multiplicities. Three focused main-success sequences with activations and returns.
Week 3 SOLID and Week 5 refactor review	SRP separates controller, service, rendering and storage. DIP uses RentalOperations and StorageHandler. Keep DOM/localStorage calls out of business rules.
Week 4 GRASP lecture and lab	Controller receives requests. RentalService creates reservations (Creator). Apartment quotes prices, DateRange checks overlaps and Reservation controls cancellation (Information Expert).
Weeks 5-6 patterns and storage factory lab	Factory Method selects local or memory storage. The service uses one common contract; only local storage promises survival after reload.
Week 7 structural patterns lecture/lab	Decorator adds optional logging through the same service interface. The wrapper forwards all six operations without changing their meaning.

Planned checks, not completed implementation tests: adjacent stays allowed; overlapping stays rejected; wrong-owner cancellation rejected; cancellation on check-in rejected; cancelled stays release dates; failed save leaves state unchanged; JSON reload restores working domain instances; decorator delegates once and preserves outcomes.

Sources: Valon Raca, SWEN 383, supplied Week 2-7 lectures and Wednesday labs, including the Week 5 W3/W4 review guide. Detailed filenames are listed in docs/course-traceability.md. Submission requirements follow the instructor reminder supplied on 22 October 2026.